Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

DESARROLLO DE APLICACIONES MÓVILES

Guía Práctica N.º 13

Proyecto integrador: refinamiento offline-first, Firebase y permisos nativos

Dato	Detalle
Estudiante	Julio Fabián Rodríguez Bazán
Docente	Benjamin David Reyna Barreto
Ciclo / Semestre	VI · 2026-2
Unidad / Sesión	Unidad 3 · Sesión 13
Proyecto	PharmaMobil (KMP) sobre PharmaSoft
Repositorio	github.com/fabianrodriguez33/Pharma-Mobil
Rama	feature/integrador-13
Fecha	4 de noviembre de 2026

1. Resumen de lo realizado

Se convirtió la aplicación PharmaMobil en un producto entregable: la cola de sincronización tiene reintentos acotados con espera creciente, el estado de la sincronización se ve en la pantalla principal, el permiso de notificaciones se resuelve desde el código común con sus tres estados y Firebase se integra detrás de una interfaz del dominio. Se trabajó en la rama **feature/integrador-13** con un commit por paso.

Paso	Qué se hizo	Commit
1	Recorrido del flujo y tabla de huecos con decisión	docs(s13): tabla de huecos del recorrido completo...
2	Columnas intentos y ultimoError, migración 2.sqm y consultas de la cola	feat(s13): columnas intentos y ultimoError con migracion...
3	SincronizarUseCase con ResultadoEnvio: transitorio o permanente, espera 1-16 s, máximo 5	feat(s13): sincronizacion con espera creciente...
4	Pendientes, fallidas y antigüedad en el Catálogo; distintivo por fila	feat(s13): mostrar pendientes, fallidas y antiguedad...
5	GestorPermisos común, implementación Android y tarjeta de notificaciones	feat(s13): gestor de permisos en comun...
6	Telemetria (dominio) + TelemetriaFirebase (datos) y evento de sincronización	feat(s13): telemetria con Firebase detras de la interfaz...
7	Pruebas de commonTest: sincronización, permisos y textos	test(s13): pruebas de sincronizacion, permisos...

2. Paso 1 · Recorrido del flujo y huecos

Recorrido: instalación desde cero, inicio de sesión, creación de un registro, pérdida de conexión, sincronización, error de validación y cierre de sesión.

N.º	Momento	Qué se observó	Grav.	Decisión
1	Primera apertura, sin sesión	Solo se ve el login; tiene salida.	Baja	Sin hueco
2	Credenciales incorrectas	El mensaje distingue credencial inválida de fallo de red.	Baja	Sin hueco
3	Listado con base vacía	Estado vacío sin acción y con texto que parece error.	Media	Se cierra hoy: texto propio y botón Sincronizar ahora
4	Creación en modo avión	Se guarda pendiente pero el catálogo no la distingue.	Alta	Se cierra hoy: distintivo por fila y contador
5	Sincronización al recuperar señal	Un pendiente que falla se reintentaba sin límite ni memoria.	Alta	Se cierra hoy: intentos, espera creciente, fila fallida visible
6	Error de validación del servidor	El mensaje llega al campo (sesión 8).	Baja	Sin hueco
7	Token vencido en un formulario	Se vuelve al login y se pierde lo escrito.	Media	Sesión 14: guardar borrador
8	Cierre de sesión	El token se borra, pero la base local conserva datos del usuario anterior.	Media	Sesión 14: avisar si hay pendientes antes de limpiar
9	Permisos del sistema	No se resolvía ningún permiso.	Alta	Se cierra hoy: GestorPermisos
10	Observabilidad	Un fallo en el teléfono no deja rastro.	Media	Se cierra hoy: Telemetria con Firebase
11	Creación reintentada tras tiempo agotado	Si el servidor guardó pero la respuesta se perdió, el reintento duplica.	Alta	Sesión 14: requiere clave de operación en PharmaSoft

Prioridad de hoy: (1) cola de pendientes acotada y visible; (2) permiso de notificaciones con sus tres estados; (3) estado vacío con acción y telemetría.

3. Paso 2 · Base local con el estado de envío

La tabla de la guía se llama *labor*; en este proyecto es *producto*. La migración es **2.sqm** porque 1.sqm ya existía (tabla de sincronización). El esquema final queda en Producto.sq.

```
-- 2.sqm
ALTER TABLE producto ADD COLUMN intentos INTEGER NOT NULL DEFAULT 0;
ALTER TABLE producto ADD COLUMN ultimoError TEXT;

-- Producto.sq (consultas nuevas)
pendientesPorEnviar: SELECT * FROM producto WHERE pendiente = 1
                     AND intentos < :maxIntentos ORDER BY id DESC;
contarPendientes:    SELECT COUNT(*) ... pendiente = 1 AND intentos <  :maxIntentos;
contarFallidas:      SELECT COUNT(*) ... pendiente = 1 AND intentos >= :maxIntentos;
sumarIntento:        UPDATE producto SET intentos = intentos + 1, ultimoError = ? WHERE id = ?;
marcarFallida:       UPDATE producto SET intentos = :maxIntentos, ultimoError = :motivo WHERE id = :id;
reintentarFallidas:  UPDATE producto SET intentos = 0, ultimoError = NULL WHERE pendiente = 1
                     AND intentos >= :maxIntentos;
```

4. Paso 3 · Sincronización refinada

La traducción de errores a **ResultadoEnvio** ocurre en la capa de datos (EnvioRepositorioRest). El caso de uso queda sin ninguna referencia a códigos HTTP.

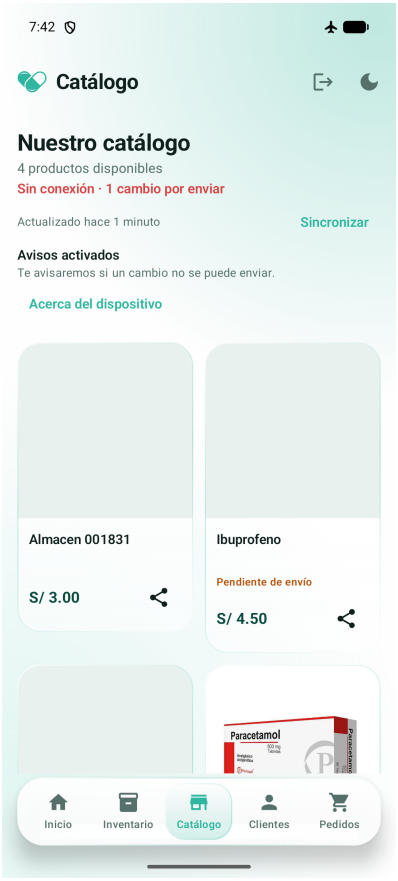
Error de la API	Resultado	Reintento
SinConexion, TiempoAgotado, Servidor (503)	Transitorio	Sí, con espera creciente
Validacion (400), NoEncontrado (404), Conflicto (409), SinPermiso (403), NoAutenticado (401)	Permanente	No: la fila se marca y se muestra

```
class SincronizarUseCase(local, remoto, esperar = { delay(it) }) {
  suspend operator fun invoke(conEspera: Boolean = true): ResumenSync {
    local.pendientes(MAX_INTENTOS).forEach { fila ->
      when (val r = remoto.enviar(fila.producto)) {
        is Exito      -> local.confirmar(fila.id, r.idServidor)
        is Permanente -> local.marcarFallida(fila.id, r.motivo, MAX_INTENTOS)
        is Transitorio -> { local.sumarIntento(fila.id, r.motivo)
                            if (conEspera) esperar(espera(fila.intentos)) }
      }
    }
    // Solo despues de subir se baja: invertirlo borra lo pendiente.
    local.conciliar(remoto.listar())
    return ResumenSync(enviadas, fallidas, local.contarPendientes(MAX_INTENTOS))
  }
  fun espera(intentos: Int) = 1_000L shl intentos.coerceIn(0, 4)  // 1, 2, 4, 8, 16 s
  const val MAX_INTENTOS = 5
}
```

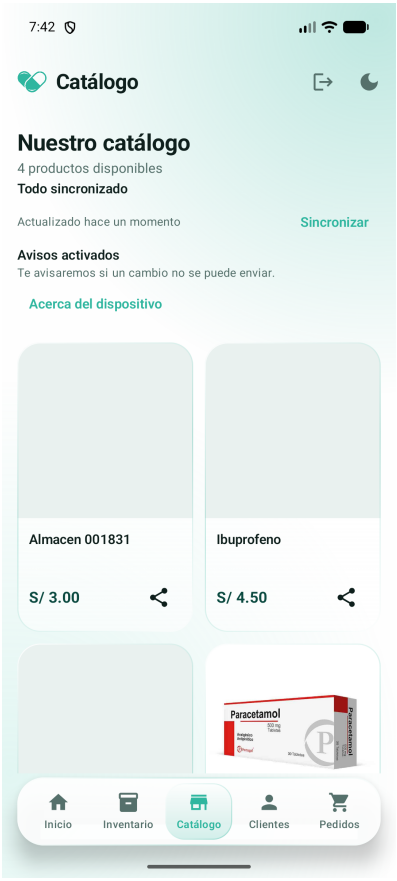
Al guardar un producto sin red la sincronización inmediata se hace con *conEspera* = *false*, para no bloquear la pantalla.

5. Paso 4 · Estado de la sincronización en la pantalla principal

El ViewModel arma **SincronizacionUiState** (enCurso, pendientes, fallidas, ultimaSync, sinConexion) combinando consultas observables de la base local; nunca consulta la red. El Catálogo muestra «Sin conexión · 1 cambio por enviar», «Actualizado hace 1 minuto», un distintivo «Pendiente de envío» por fila y, si hay filas fallidas, «1 cambio no se pudo enviar» con las acciones Revisar y Reintentar ahora.



Modo avión: 1 cambio por enviar y fila pendiente

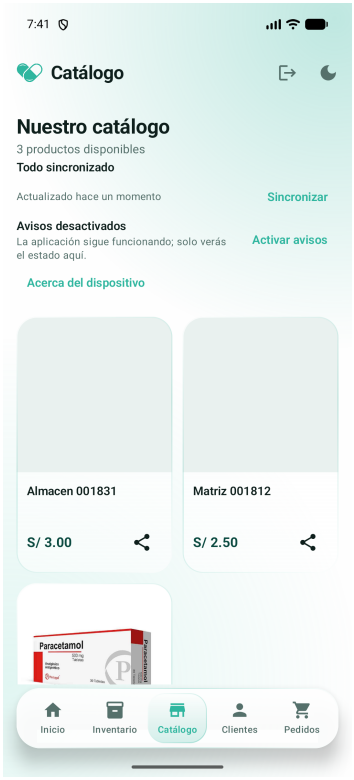


Tras reconectar y sincronizar: todo sincronizado

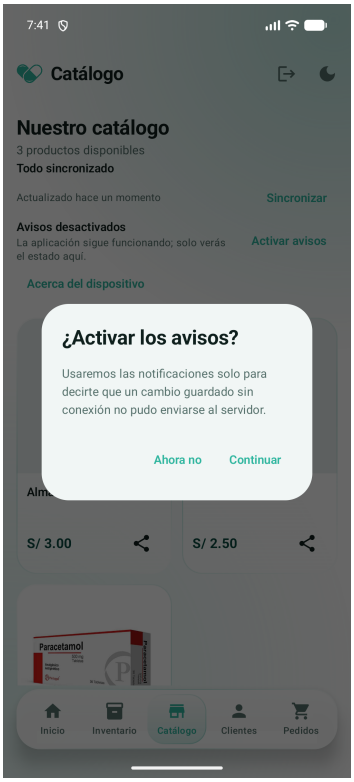
6. Paso 5 · Gestor de permisos

GestorPermisos se declara como interfaz en commonMain (estado, solicitar, abrirAjustes) con los estados Concedido, Denegado y DenegadoParaSiempre. La implementación de Android (GestorPermisosAndroid + PuentePermisos) recuerda si el permiso ya se pidió, para distinguir «primera vez» de «denegado definitivamente». Se registra con Koin en platformModule; por eso vive en shared/androidMain y no en androidApp. El manifiesto declara POST_NOTIFICATIONS, CAMERA y ACCESS_FINE_LOCATION; Info.plist incluye las claves de cámara y ubicación en español. El permiso real resuelto en la aplicación es el de **notificaciones**.

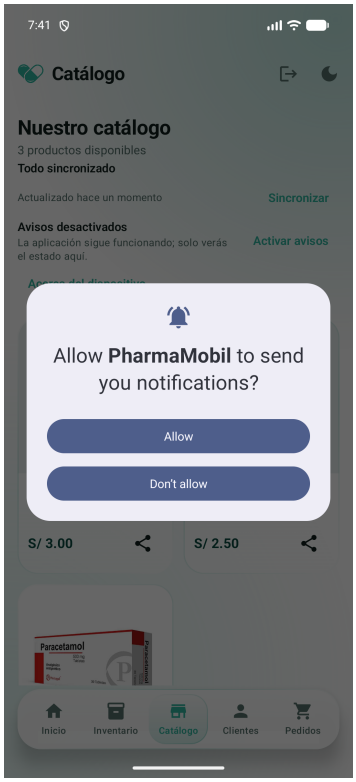
Estado	Qué ve el usuario	Cómo se recupera la función
Concedido	«Avisos activados»	Continúa normal
Denegado	«Avisos desactivados» + botón Activar avisos; antes de pedir, un diálogo explica para qué	Se vuelve a pedir tras la explicación
Denegado para siempre	«Avisos bloqueados» + diálogo «Permiso bloqueado»	Botón Abrir ajustes; al volver, el estado se refresca solo



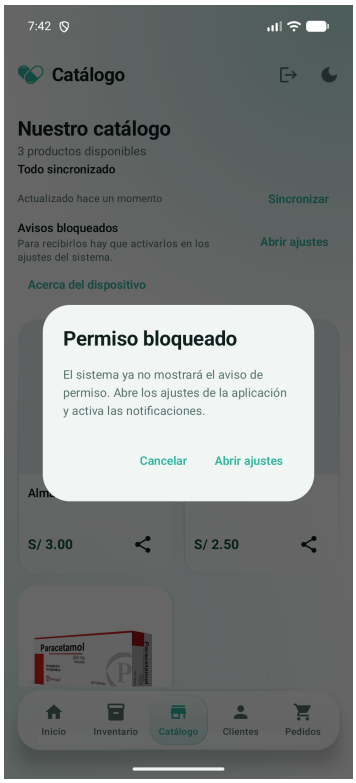
Estado inicial: denegado (primera vez)



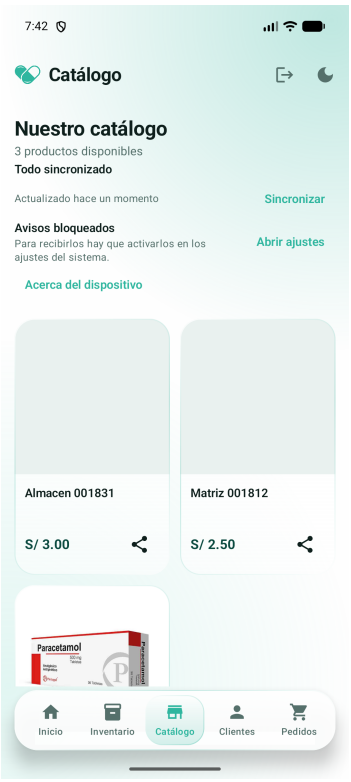
La app explica antes de pedir



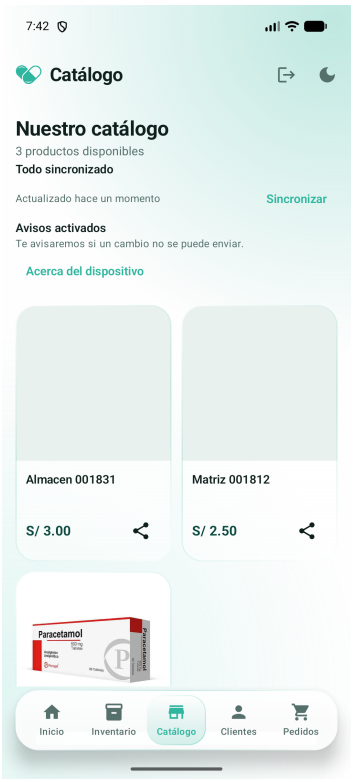
Diálogo del sistema



Segunda negativa: denegado para siempre



Tarjeta «Avisos bloqueados»

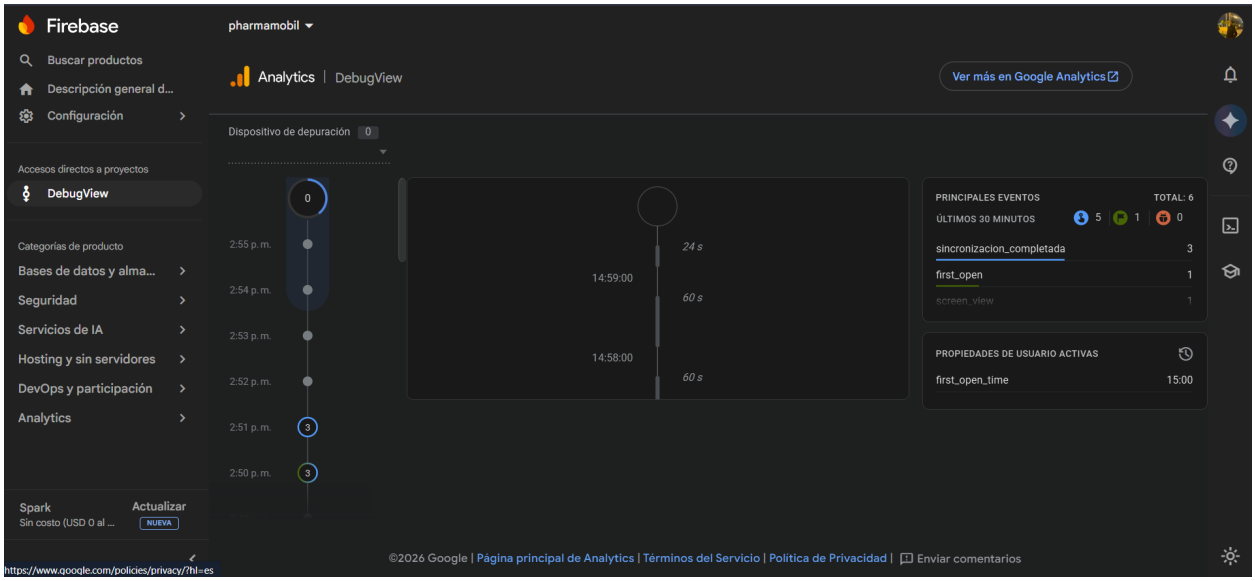


Permiso concedido

7. Paso 6 · Firebase detrás de una interfaz del dominio

El dominio declara **Telemetria** (registrarError, registrarEvento). Solo la clase **TelemetriaFirebase** (data/firebase) importa el SDK de GitLive 2.7.0 (crashlytics, analytics y messaging); se registra en Koin con *single<Telemetria> { TelemetriaFirebase() }*. Se registra únicamente contexto técnico (pantalla, operación, tipo de error), nunca datos personales ni el token.

google-services.json se descarga de la consola de Firebase y no se versiona (.gitignore). El plugin de Google Services se aplica solo si el archivo existe, de modo que la aplicación compila y corre también sin él, con la telemetría inactiva. Evento registrado: **sincronizacion_completada** (parámetro pantalla = catalogo).



Consola de Firebase, proyecto pharmamobil: Analytics, DebugView.

8. Paso 7 · Pruebas de commonTest

Se usan la base SQLite en memoria y el motor de red simulado (MockEngine); el gestor de permisos se prueba con una implementación falsa. Resultado de *./gradlew :shared:testAndroidHostTest*: **100 pruebas, 0 fallas** (20 nuevas).

Prueba	Comprueba
unFalloTransitorioSumaIntentoYNoMarcaFallida	503: intentos = 1, sigue pendiente, no es fallida
unFalloPermanenteMarcaLaFilaYNoLaReintenta	409: fila fallida y un solo POST aunque se sincronice dos veces
laFilaQueAgotaSusIntentosQuedaVisibleYDejaDeReintentarse	Tras 5 intentos queda fallida y deja de enviarse
laEsperaCreceConCadaIntentoYTieneTope	1, 2, 4, 8, 16, 16 s
laSubidaOcurreAntesQueLaBajadaYNadaSePierde	Orden POST luego GET; el pendiente sobrevive a la bajada
reintentarLasFallidasLasDevuelveALaColaYLasEnvia	La acción del usuario devuelve la fila a la cola
PermisoNotificacionesViewModelTest (6)	Concedido, denegado con explicación, ahora no, denegado para siempre, Ajustes, refresco
TextoEstadoSincronizacionTest (6)	Mensajes «Sin conexión · 3 cambios por enviar», singular, fallidas

9. Lista de cotejo

N.º	Criterio observable	Cumple
1	Recorrí el flujo completo y registré por escrito los huecos encontrados.	Sí

N.º	Criterio observable	Cumple
2	La tabla local guarda los intentos y el último error de cada fila pendiente.	Sí
3	La sincronización distingue el fallo transitorio del permanente.	Sí
4	La espera entre intentos crece y existe un número máximo de intentos.	Sí
5	Una fila que agota los intentos queda marcada y visible para el usuario.	Sí
6	La subida ocurre antes de la bajada y nada se pierde al sincronizar.	Sí
7	La pantalla principal muestra pendientes, fallidas y antigüedad de los datos.	Sí
8	El gestor de permisos está declarado como interfaz en el código común.	Sí
9	La implementación de Android distingue los tres estados del permiso.	Sí
10	El permiso está declarado en el manifiesto y, si aplica, la clave está en Info.plist.	Sí
11	La interfaz explica antes de pedir y ofrece ir a Ajustes cuando corresponde.	Sí
12	El SDK de Firebase no aparece fuera de la capa de datos.	Sí
13	Las pruebas de commonTest se ejecutan y pasan.	Sí
14	Los commits siguen la convención y están distribuidos a lo largo de la sesión.	Sí

Notas. (1) GestorPermisosIos es mínimo (solo abre Ajustes) y no se compiló en este equipo; los diálogos nativos de iOS quedan para la sesión 14 con un Mac. (2) La fila fallida se verificó con pruebas automáticas.